CST8507: Natural Language Processing

Lab 1

Part 1: IDE Setup

The Lab1 commands are demonstrated for Windows, for other OS please refer to the original documentation.

Conventions used within this document:

```
> Anaconda Command Prompt

>>> Python code

User supplied values
```

Objectives

Install Main Python Libraries for NLP

Create Your Environment

1- You can create as much environments as you want, each environment can have different Python version or even similar versions with different libraries. For example: you can create one environment for Python 3.10 using the following command:
`> conda create -n myenv python=3.10`
(name myenv py10 for a better reference)

Note: using the above command you can even install the required packages, but for simplicity we will do it step-by-step.

2- You can check the environment that you created using the following command:
`> conda env list`
3- You can activate the new environment: using the following command:
`> conda activate py10`

For conda management use the following reference:

https://conda.io/projects/conda/en/latest/user-guide/tasks/manage-environments.html

Install Important Libraries

```
> conda install numpy
> conda install -c conda-forge matplotlib
> conda install pandas
> conda install -c conda-forge statsmodels
> conda install -c anaconda scikit-learn
>conda install -c anaconda scipy

Or
conda install conda-forge::scip

We can do the steps in anaconda navigator
```

Install Main Python Libraries for NLP

We have a large collection of NLP libraries available in Python.

1. **Natural Language Toolkit (NLTK)**

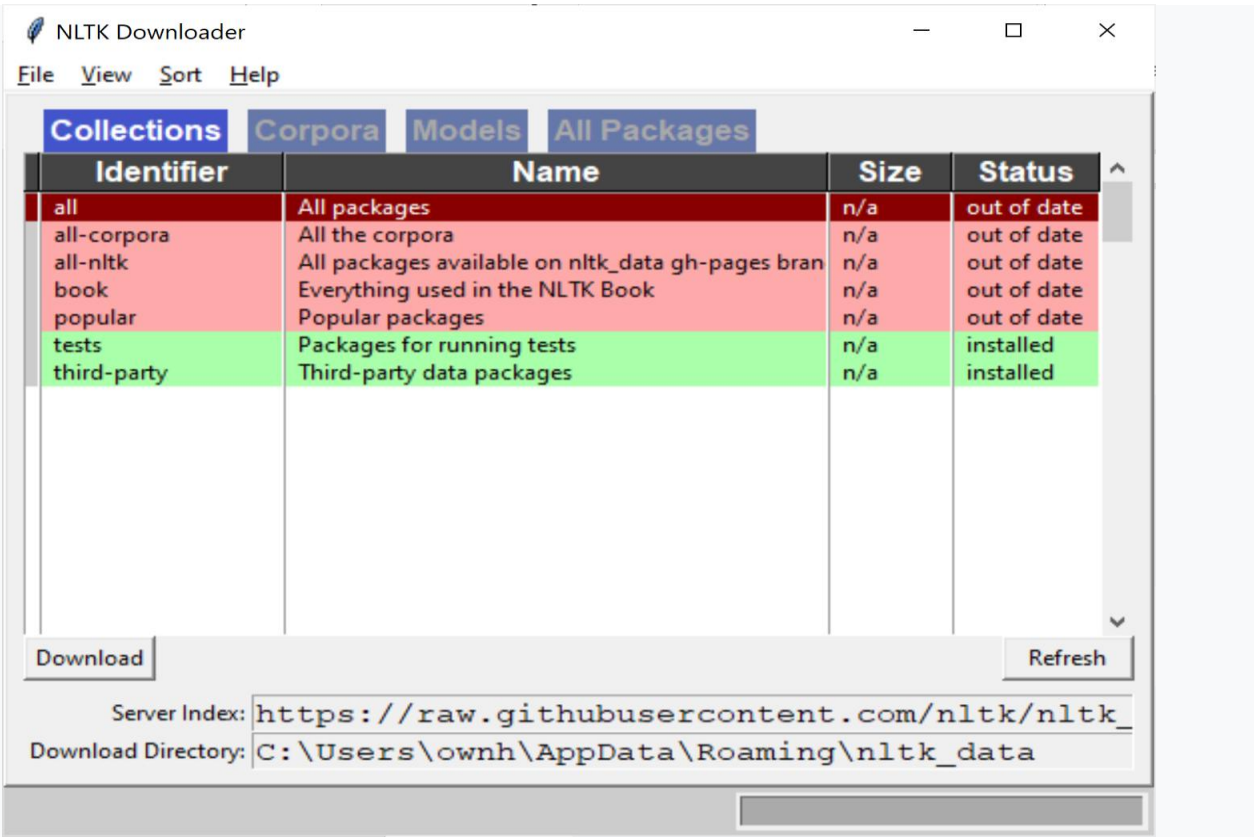
   To install NLTK and its dependencies by running the following command:

   ```
   > conda install -c anaconda nltk
   ```

   To Import package and download model type :

   ```
   > python
   >>> import nltk
   >>> nltk.download()
   ```

NLTK Downloaded Window Opens. Click the Download Button to download the dataset.



To test the installed data, use the following code

```
>>> from nltk.corpus import brown
>>> brown.words()
```

- Create a Microsoft word file, name it" Lab1.docx".
- **Take a screenshot and save in "Lab1.docx".**

Note: to exit from Python console use:

```
>>> exit()
```

2. spaCy

It is one of the most trending and advanced libraries for implementing NLP today. It is many distinct features that provide clear advantage for processing text data and modeling. To install spaCy and its dependencies by running the following command:

```
> conda install -c conda-forge spacy
> python -m spacy download en_core_web_sm
```

To Import package and download model type:

```
>python
>>> import spacy
```

To load the models and data for English Language, you have to use

```
>>> nlp = spacy.load('en_core_web_sm')
```

nlp object is referred as language model instance.

To test the installed data, use the following code

```
>>> text = ("When Sebastian Thrun started working on self-driving cars at "
        "Google in 2007, few people outside of the company took him "
        "seriously. "I can tell you very senior CEOs of major American "
        "car companies would shake my hand and turn away because I wasn't "
        "worth talking to," said Thrun, in an interview with Recode earlier "
        "this week.")
>>> doc = nlp(text)
>>> print("Noun phrases:", [chunk.text for chunk in doc.noun_chunks])
>>> print("Verbs:", [token.lemma_ for token in doc if token.pos_ == "VERB"])
```

- **Take a screenshot and save in "Lab1.docx".**

Part 2: Zipf's Law and Text Analysis

Objectives

- Preprocess raw text for NLP analysis
- Compute and analyze word frequency distributions
- Empirically test Zipf's Law on real-world data

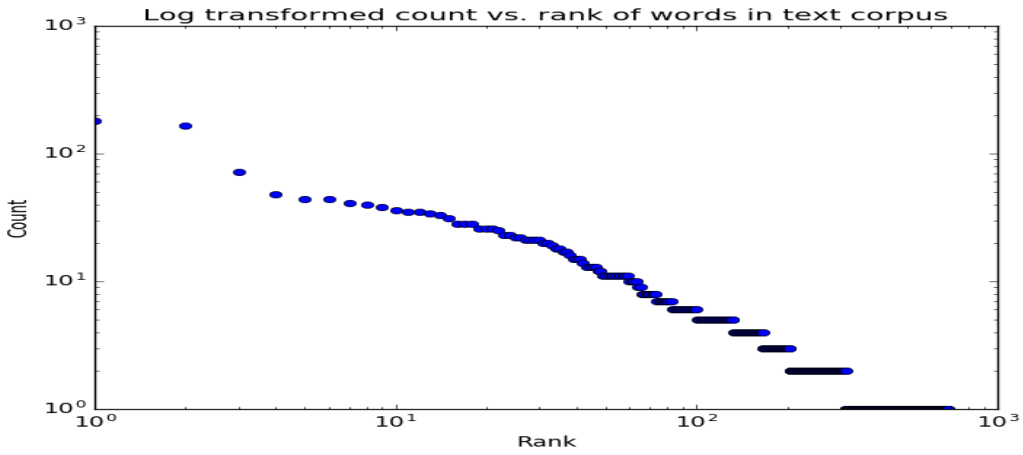
Learning Resources

Lecture Slides and resources including Hybrid work (week 1,2).

Background

About 80 years ago, George Kingsley Zipf reported an observation that the frequency of a word seems to be a power law function of its frequency rank, formulated as f(r) $\propto r\alpha$, where f is word frequency, r is the rank of frequency, and α is the exponent(1, 2). This linguistic regularity was later termed as Zipf's law. For almost any corpus the frequency of the occurrence of a word (i.e., how many times it occurs) is inversely proportional to the word's frequency rank in the corpus, i.e. For example, the most frequent word (rank of 1) generally occurs twice as many times as the second most frequent word (rank of 2), etc.

As you see in the lecture, Zipf's law is most easily observed by plotting the data on a log-log plot. In this form we would expect a linear relationship of the form.



In this lab, you will empirically investigate Zipf's Law using real text data. The goal is not only to confirm the law, but to understand how **text preprocessing choices, genre, and linguistic structure affect word frequency distributions**.

Steps:

Step 1:Import the required libraries, for example:

```
import string
from collections import Counter
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import nltk

from nltk.tokenize import word_tokenize
from nltk.probability import FreqDist
```

Step2: Data selection

You must select two English text datasets:

- **Literary text**
  Examples: novel chapter, short story collection, play, or public-domain literature
- **Informational / non-literary text**
  Examples: news articles, Wikipedia pages, technical blogs, reports

- Each text must contain at least 5,000 words after cleaning to show a clear plot.

For example, you can use NLTK library to import **`gutenberg`** .
The Gutenberg corpus is a collection of books that are in the public domain and can be freely used for natural language processing tasks.

```
from nltk.corpus import gutenberg

print('The following books exist in gutenberg :')
gutenberg.fileids()

#This will print the list of books

# After executing this statement, the variable text will contain the ent
#ire raw text content of "Emma", allowing you to perform various text pr
#ocessing tasks or analyses on it using Python.

text = gutenberg.raw('austen-emma.txt')
```

gutenberg.raw('austen-emma.txt') is a function from the Natural Language Toolkit (NLTK) library in Python that is used to access the raw text of a book from the NLTK's Gutenberg corpus.
The function will return the raw text of the book "Emma" by Jane Austen **as a single string.**

Step 3: Tokenizing the data

Tokenization is a common pre-processing step in natural language processing (NLP) tasks such as text analysis, text mining, and machine learning. Tokenization can help to make text data more manageable and easier to work with by breaking it down into smaller, more manageable pieces. For example, once text is tokenized, it is much easier to perform operations such as counting the frequency of words, removing stop words, or creating a Zipf's Law plot.

```
tokens = word_tokenize(text)
```

Step 4: Empirical verification of Zipf's Law

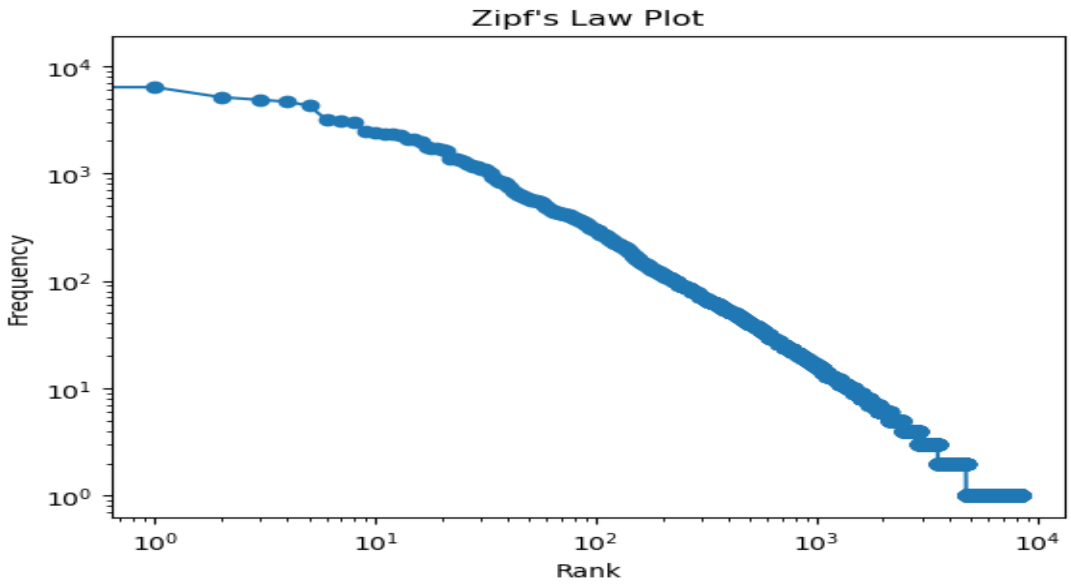
For each text, perform the following:

- Compute word frequencies
- Rank words from most frequent to least frequent
- Calculate and print:
    - Top 20 most frequent words
    - Total vocabulary size
    - Plot **word rank vs frequency** using a **log–log scale**.

```
fdist = FreqDist(tokens)

sorted_fdist = sorted(fdist.items(), key=lambda x: x[1], reverse=True)
```

Your output graph may look like the following



Step 5: Comparative analysis

Write and submit a comparative discussion that compare the two texts with respect to:

- Shape of the Zipf curve
- Steepness of the frequency decay
- Vocabulary richness and long-tail behavior

Add your discussion to *Lab1.docx* (section title: *'Step 5: Comparative Analysis'*).

Step 6: Evaluating the Stability of Zipf's Distribution

To test the limitations of Zipf's Law, the following analysis was repeated on the one for **one text**:

- **Remove stopwords**
  Create a new log–log plot and write a discussion on of how and why the distribution changed.

- **Restrict analysis to a single part of speech (e.g., nouns only)**

  Create a new log–log plot and write a discussion on of how and why the distribution changed.

Add your discussion to Lab1.docx (section title: 'step 6: Evaluating the Stability of Zipf's Distribution).

Code design and style Requirements.

Make sure that your program is properly documented:

- You should have a docstring at the very beginning of the file briefly describing your program and stating your name, section, and creativity additions.
- Each function should have an appropriate docstring (including arguments and return value if applicable).
- Other miscellaneous comments to make things clear.

In addition, make sure that you have used good code design and style (including meaningful variable names, constants where relevant, vertical white space, etc.).

Submission Instruction

Submit your code **as a Jupyter Notebook with the running code**, you can do the following steps:

1. Open your Jupyter Notebook: You can open Jupyter Notebook either from the command line by typing "jupyter notebook" or from Anaconda Navigator.
2. Write your code: Write the code you want to submit in the cells of the Jupyter Notebook. **Make sure to run the cells so that the output is generated**.
3. Save the Notebook: Once you have written and run your code, save the Jupyter Notebook by clicking on File -> Save and Checkpoint or by pressing "Ctrl + S".
4. Export the Notebook: To export the Jupyter Notebook, go to File -> Download as -> Notebook (.ipynb). This will download the Notebook as a .ipynb file on your laptop.

Submitting Your Lab Files

1. Create a Folder and name the folder lab1.
    - Place your program file, named lab1_zipf_law, inside the lab1 folder.
    - Place your document file, Lab1.docx , inside the lab1 folder.
2. Zip the entire lab1 folder.
3. Upload the zipped file to **Brightspace**.

Ensure your files are correctly named and organized before submission.

You can submit multiple times, with only the most recent submission (before the due date) graded.

Due Date

Check the Brightspace.

Grading Criteria

Criterion	10 points	5 points	0 points
Data selection	Texts are well-chosen, distinct genres, >5000	Texts meet requirements <5000	Texts not meeting requirements
Tokenization	Correctly applied	Some minor errors	Incorrect
Word frequency analysis	Accurate frequency counts, rankings, top words, vocabulary size	Minor errors in analysis	Analysis missing
Empirical verification of Zipf's Law	Correct log–log plots, labeled, Zipf exponent estimated, clear explanation	Correct plots but interpretation limited	Plots missing, incorrect, or misinterpreted
Comparative analysis	Insightful comparison, explains why differences occur between texts	Comparison present but not complete	Comparison missing or incorrect
Evaluating the Stability of Zipf's Distribution	Modification applied correctly, Clear explanation is provided	Modification done but explanation is missing	Experiment missing or incorrect
Submission	Follow submission instructions	Missing some submission instructions	Not follow the submission instruction